TFG · SONDA PASIVA DE DETECCIÓN PQC

El TFG de Celia Pascual como base de sonda

Qué arquitectura reutilizar de un analizador pasivo en C con libpcap, y qué errores evitar

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **12/10/2026** | Tutoría del miércoles 14/10/2026

1. Qué es y por qué sirve de referencia

Desarrollo de un sistema eficiente de análisis de tráfico ModBus TCP para la detección de anomalías en redes SCADA. Enero de 2020, mismo tutor. Es un IDS en C con libpcap que analiza tráfico Modbus TCP.

El protocolo no tiene nada que ver, y da igual: lo aprovechable es **la arquitectura de una sonda pasiva en C**, que es la misma se analice Modbus o TLS.

Hay además un precedente que justifica la elección de lenguaje

En su motivación (pág. 1) escribe:

> *"Para la realización de este proyecto se ha tomado como referencia otro TFG [5] desarrollado en Python, razón por la cual no procesaba el tráfico a una velocidad suficientemente rápida para considerarse eficiente. Por este motivo, el lenguaje elegido para el desarrollo de este TFG ha sido C."*

Ese [5] es el TFG de Culebras Sánchez (2016). Su bibliografía incluye además [8], Sanches Gómez (2019), que hizo lo equivalente para IEC 60870-5-104.

> **LECTURA**
>
> El grupo lleva años encadenando TFGs sobre sondas pasivas de protocolos, y existe un **precedente documentado de migrar de Python a C por rendimiento**, dirigido por el mismo tutor. Es el argumento que respalda la elección de lenguaje de este TFG, y no hay que construirlo: ya está escrito. Este trabajo continúa al de Roberto Sanz igual que el de Celia continuó al de Culebras.

2. La arquitectura: el esqueleto reutilizable

```
main()
 ├── getopt: -i interfaz  ó  -f traza.pcap
 ├── pcap_open_live()  ó  pcap_open_offline()
 ├── pcap_loop(descr, -1, analizar_paquete, NULL)
 └── al salir: resumen + tablas + medidas de rendimiento

analizar_paquete()          <- una vez por paquete
 ├── Ethernet  -> ¿VLAN? -> ajusta el offset acumulado
 ├── IP        -> filtros: version 4, IHL>=20, offset=0, TTL>1
 ├── TCP       -> filtros: protocolo 6, cabecera >=20
 └── Modbus    -> filtros: puerto 502, protocol_id=0, longitud valida
                  └── bucle: varias PDU Modbus en un mismo paquete
```

Su modelo es de **filtros en cascada**: el paquete atraviesa las capas y en cada una o se descarta o se extraen campos. La correspondencia con esta sonda es directa:

Celia (Modbus)	Esta sonda (TLS)
Filtro puerto 502	Filtro puerto 443
Cabecera Modbus (MBAP)	Cabecera de registro TLS
Campo `function_code`	`ClientHello` / `ServerHello`
Varias PDU Modbus por paquete	Varios registros TLS por segmento
Tabla hash indexada por IP	Tabla hash indexada por 5-tupla

La penúltima fila es especialmente útil: ya resolvió el caso de **varias unidades de protocolo dentro de un mismo paquete**, con un bucle que avanza mientras quede payload. Ese patrón sirve tal cual para los registros TLS.

Qué copiar

- El esqueleto de `main`: `getopt`, las dos fuentes de entrada, `pcap_loop`, manejo de `SIGINT`.
- Los structs de cabecera mapeados sobre el buffer y las macros `IP_HL()` y `TH_OFF()`.
- El concepto de offset acumulado, con la etiqueta VLAN sumando bytes.
- **Las medidas de rendimiento finales**: tiempo de ejecución, paquetes por segundo y throughput. Imprescindibles para comparar contra Zeek.
- El bucle sobre varias PDU en un mismo paquete.

3. Qué NO copiar

Revisado su `analisisModbus.c`, contiene errores reales. Es un TFG, no código de producción, y la mayoría no se manifiestan con trazas pequeñas.

3.1. No hay reensamblado TCP

Lee cada paquete de forma independiente. Puede permitírselo porque **una PDU Modbus cabe siempre en un paquete**. Un `ClientHello` postcuántico no. Es la diferencia estructural entre su sonda y esta, y el núcleo técnico de este TFG.

3.2. Tres de las cuatro funciones de inserción tienen la lista rota

```
// insert_IP() — falta el ';' o las llaves
else{
    for(n1=hashTable[key]; n1->next != NULL; n1=n1->next)
    n1->next = new_node;          // <-- es el CUERPO del bucle
}
```

La asignación es el cuerpo del bucle, no lo que viene después. Si la cadena tiene un solo nodo el bucle no se ejecuta y **el nuevo nodo nunca se enlaza**; si tiene más, se enlaza pero **se pierde el resto de la cadena**.

El mismo error está en `insert_ID()` y `insert_peticion()`. En `insert()` —la de `DataNode`— sí está bien, con el `;` al final del `for`. Cuatro funciones casi idénticas, y en tres se le escapó. No se manifestó porque con pocas IP apenas hay colisiones.

3.3. `printlist()` es un use-after-free

```
for(n1=n; n1!=NULL; n1=n1->next){
    printf(...);
    free(n1);                     // <-- libera
}                                 //     y el incremento lee n1->next
```

Libera el nodo y acto seguido el incremento del bucle lee memoria liberada.

3.4. El suavizado exponencial comparte estado entre clientes

```
double media = 0.2;          // GLOBAL
double desviacion = 0.1;     // GLOBAL

void smoothing(){
  for(...){ for(n=...; n!=NULL; n=n->next){
      media = (alpha*(n->num_pet) + (1-alpha)*media);
      n->media = media;        // guarda por nodo, pero acumula en la global
```

Guarda la media **por nodo** pero la variable que acumula es **global**, de modo que la media de un cliente arrastra la del anterior. La detección de anomalías por tasa queda contaminada entre dispositivos. En esa misma línea usa `abs()` sobre un `double`, que trunca la parte decimal: debería ser `fabs()`.

3.5. Las funciones hash son sumas de octetos

```
key = ip_src[0]+ip_src[1]+ip_src[2]+ip_src[3];
```

Valor máximo 1020 sobre una tabla de 10000 posiciones: **el 90 % de la tabla no se usa nunca** y las IP cuyos octetos suman lo mismo colisionan. Ella misma lo deja como trabajo futuro (pág. 38).

Además declara `DataNode *hash[1000000]` con `hashSize = 10000`: casi un millón de punteros reservados sin usar. Y `search()` indexa con la clave sin aplicar el módulo mientras `insert()` sí lo aplica; funciona solo porque la clave nunca alcanza 10000.

> **DECISIÓN PARA ESTA SONDA**
>
> Usar una función hash de calidad sobre la **5-tupla completa** (FNV-1a o equivalente, cuatro líneas de código) y dimensionar la tabla de acuerdo con el número de entradas esperado. Con eso desaparece el problema entero.

4. Sus resultados: la línea base de rendimiento

Medida	Resultado
Traza real	~6 millones de paquetes en 24 h, cedida por la Cátedra UAM-Naudit
Retransmisiones	**18 %**, que tuvo que filtrar a mano porque su IDS no las detecta
Tiempo de ejecución	0,97 s sobre 5 M de paquetes
Paquetes por segundo	**5,2 Mpps**
Throughput	**2,7 Gbps** (objetivo inicial del TFG: 1 Gbps)

Ese 2,7 Gbps es la vara de medir para el capítulo de evaluación de este TFG. Y el 18 % de retransmisiones es un aviso: **aquí no se pueden filtrar a mano**, hay que detectarlas, porque forman parte del reensamblado.

> **UNA LECCIÓN DE SU VALIDACIÓN**
>
> La traza real fue la que le obligó a añadir el soporte de VLAN —ninguna traza anterior la llevaba— y la que destapó que su detección de maestros fallaba con más de uno. **Las trazas de laboratorio no encuentran los bugs; las reales sí.** Conviene plantear al tutor el acceso a una traza real para la validación de esta sonda.

Código fuente: `github.com/celiapascual/modbustcp`

5. Resumen para la tutoría

Qué se aprovecha	Qué se descarta
`pcap_loop` con callback y doble fuente de entrada	El modelo «un paquete, un mensaje»
Filtros en cascada por capas	Hash por suma de octetos de IP
Tablas hash con listas enlazadas (el concepto)	Su implementación de inserción, rota en 3 de 4 casos
Bucle sobre varias PDU por paquete	Indexado por IP suelta en lugar de por flujo
Medidas de rendimiento al final	El suavizado con estado global compartido
El precedente Python → C por rendimiento	—